App Dev Project Report

Project Report — IITM BS Degree

1. Student Details

Name: Divyansh Shekhar
Roll Number: 23F2004203
Email: 23f2004203@ds.study.iitm.ac.in

About Me: I am a Diploma level student in IITM BS Degree with huge interest in Building Agent Models.

2. Project Details

2.1 Problem Statement

Hospitals need efficient systems to manage patients, doctors, appointments, and treatments. Manual registers and disconnected software make it difficult to manage records, avoid scheduling conflicts, and track patient history. The objective is to build a Hospital Management System (HMS) web application allowing Admins, Doctors, and Patients to interact with the system based on their roles.

2.2 Approach & Architecture

The system was designed as a decoupled SPA with a REST API backend:

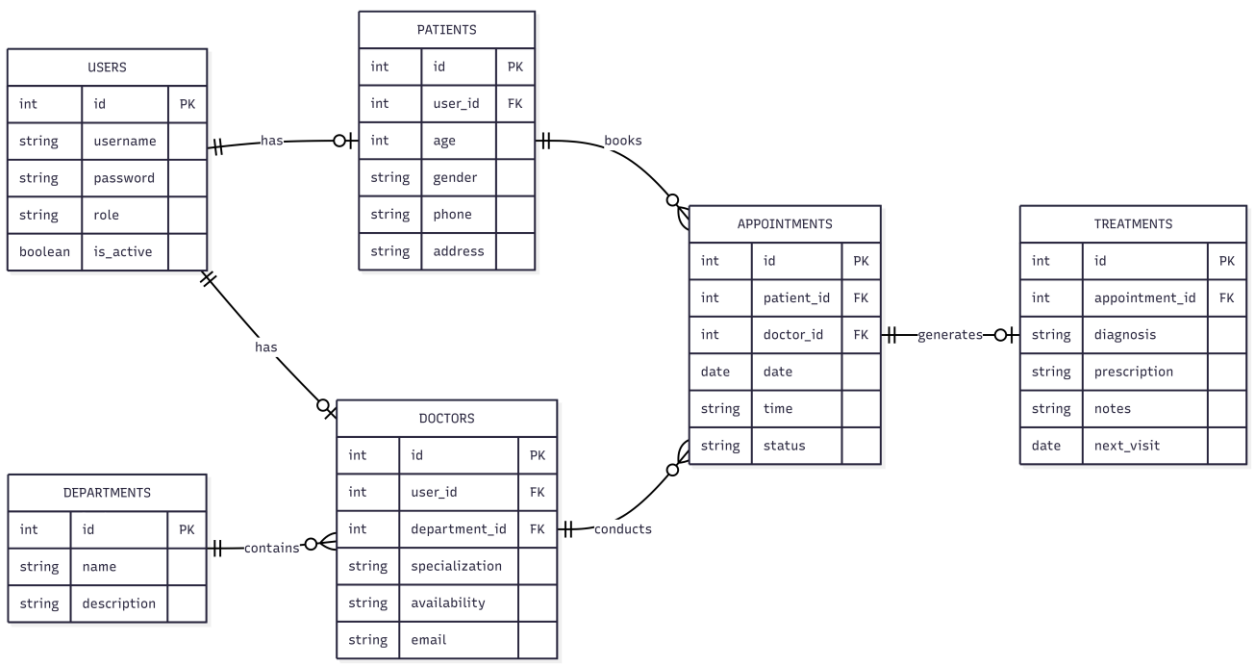
- Role-based architecture with three distinct user types: Admin, Doctor, and Patient.
- JWT-based stateless authentication — tokens issued on login, verified on every API request via Flask-JWT-Extended.
- Vue 3 SPA (Vite) as the frontend; each role gets its own layout and protected routes via Vue Router guards.
- Flask REST API backend with Blueprints separating Admin, Doctor, Patient, and Auth routes.
- SQLite database managed via SQLAlchemy ORM and Flask-Migrate for schema migrations.
- Redis (Memurai on Windows) used for response caching (with expiry) and as Celery's broker & result backend.
- Celery handles three async/scheduled jobs: daily appointment reminders (Google Chat), monthly doctor reports (email), and patient CSV export (user-triggered).
- Bootstrap 5 for responsive UI across all device sizes.

3. Frameworks and Libraries Used

Library / Framework	Layer	Purpose
Flask 3.x	Backend	Core REST API framework
Flask-JWT-Extended	Backend	JWT authentication & role-based access
Flask-SQLAlchemy	Backend	ORM for SQLite database
Flask-Migrate	Backend	Database schema migrations (Alembic)
Flask-Caching	Backend	Redis-backed response caching with expiry
Flask-CORS	Backend	Cross-origin requests from Vue frontend
Celery 5.x	Backend	Async & scheduled task queue
Redis / Memurai	Backend	Celery broker, result backend, cache store
SQLite	Database	Lightweight relational database
Vue 3 (Vite)	Frontend	SPA framework with Composition API
Vue Router 4	Frontend	Client-side routing & navigation guards
Bootstrap 5	Frontend	Responsive UI components & grid
Axios	Frontend	HTTP client for API calls
Werkzeug	Backend	Password hashing & WSGI utilities
python-dotenv	Backend	Environment variable management
Requests	Backend	Google Chat webhook & HTTP calls

---

4. ER Diagram



The database consists of the following key tables and relationships:

- **users —** users

  Columns: id, username, password (hashed), role (admin/doctor/patient), is_active

- **patients —** patients

  Columns: id, user_id (FK → users), age, gender, phone, address

- **doctors —** doctors

Columns: id, user_id (FK → users), department_id (FK → departments), specialization, availability (JSON), email

- **departments —** departments

Columns: id, name, description

- **appointments —** appointments

Columns: id, patient_id (FK → patients), doctor_id (FK → doctors), date, time, status (Booked/Completed/Cancelled)

- **treatments —** treatments

Columns: id, appointment_id (FK → appointments), diagnosis, prescription, notes, next_visit

---

5. API Resource Endpoints

5.1 Authentication (/auth)

Method	Endpoint	Description
POST	/auth/login	Login for all roles — returns JWT access token
POST	/auth/register	Patient self-registration
GET	/auth/me	Fetch current logged-in user info

5.2 Admin (/admin) — JWT + admin role required

Method	Endpoint	Description
GET	/admin/dashboard	Stats: total doctors, patients, appointments, departments
POST	/admin/doctors	Add a new doctor (username, password, specialization, department, email)
GET	/admin/doctors	List all doctors (paginated)
GET	/admin/doctors/<id>	Get single doctor details
PUT	/admin/doctors/<id>	Update doctor profile
DELETE	/admin/doctors/<id>	Blacklist doctor (sets is_active=False)
PUT	/admin/doctors/<id>/restore	Restore blacklisted doctor
GET	/admin/doctors/search?q=	Search doctors by name / specialization / department
PUT	/admin/doctors/<id>/availability	Override doctor availability
GET	/admin/patients	List all patients (paginated)
GET	/admin/patients/<id>	Get single patient details
PUT	/admin/patients/<id>	Update patient info
DELETE	/admin/patients/<id>	Blacklist patient
PUT	/admin/patients/<id>/restore	Restore blacklisted patient
GET	/admin/patients/search?q=	Search patients by name / phone / ID
GET	/admin/appointments	List all appointments (filterable by doctor, patient, date, status)
PUT	/admin/appointments/<id>/status	Update appointment status
POST	/admin/departments	Create department
GET	/admin/departments	List all departments
GET	/admin/departments/<id>	Get department with doctors
PUT	/admin/departments/<id>	Update department
DELETE	/admin/departments/<id>	Delete department (only if no doctors)
GET	/admin/departments/search?q=	Search departments by name

5.3 Doctor (/doctor) — JWT + doctor role required

Method	Endpoint	Description
GET	/doctor/availability	Get own availability schedule
PUT	/doctor/availability	Set own availability for next 7 days
GET	/doctor/appointments/today	List today's appointments
GET	/doctor/appointments/week	List appointments for current week
GET	/doctor/appointments	List all appointments (all time)
PUT	/doctor/appointments/<id>/status	Mark appointment Completed or Cancelled
POST	/doctor/appointments/<id>/treatment	Add treatment record (diagnosis, prescription, notes, next_visit)
PUT	/doctor/treatments/<treatment_id>	Update existing treatment record
GET	/doctor/patient/<patient_id>/history	View full history of a patient
GET	/doctor/patients	List all patients assigned to this doctor

5.4 Patient (/patient) — JWT + patient role required

Method	Endpoint	Description
GET	/patient/me	Get own profile
PUT	/patient/me	Update own profile
GET	/patient/next-visits	Get pending follow-up visit reminders
GET	/patient/departments	List all departments (public, optional auth)
GET	/patient/departments/<id>/doctors	List doctors in a department with availability
GET	/patient/doctors/<id>/availability	Get a specific doctor's availability + booked slots
GET	/patient/doctors/search?q=	Search doctors by name / specialization
POST	/patient/appointments	Book a new appointment
PUT	/patient/appointments/<id>/cancel	Cancel an appointment
PUT	/patient/appointments/<id>/reschedule	Reschedule an appointment to a new date/time
GET	/patient/appointments/upcoming	List upcoming appointments
GET	/patient/appointments/history	Past appointment history with treatment details
POST	/patient/export_treatments	Trigger async CSV export of treatment history
GET	/patient/export_treatments/status/<task>	Poll export task status
GET	/patient/export_treatments/download	Download the generated CSV file

---

6. Presentation Video

https://drive.google.com/drive/folders/1Cjjqy4xB6NsL4JTxvNvUJF6PsJYVvmKf?usp=sharing

The video covers: application overview, role-based demo (Admin / Doctor / Patient flows), backend job demonstrations (daily reminder, monthly report, CSV export), and key design decisions.